

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа № 1
За седьмой семестр
По дисциплине «Обработка изображений в интеллектуальных системах»
Тема: «Обучение классификаторов средствами библиотеки PyTorch»

Выполнил:
Студент 4 курса
Группы ИИ-26
Черноиван А.Н.
Проверил:
Андренко К.В.

Цель работы: научиться конструировать нейросетевые классификаторы и выполнять их обучение на известных выборках компьютерного зрения.

Общее задание

1. Выполнить конструирование своей модели СНС, обучить ее на выборке по заданию (использовать **torchvision.datasets**). Предпочтение отдавать как можно более простым архитектурам, базирующимся на базовых типах слоев (сверточный, полносвязный, подвыборочный, слой нелинейного преобразования). Оценить эффективность обучения на тестовой выборке, построить график изменения ошибки (matplotlib);
2. Ознакомьтесь с state-of-the-art результатами для предлагаемых выборок (из материалов в сети Интернет). Сделать выводы о результатах обучения СНС из п. 1;
3. Реализовать визуализацию работы СНС из пункта 1 (выбор и подачу на архитектуру произвольного изображения с выводом результата);
4. Оформить отчет по выполненной работе, загрузить исходный код и отчет в соответствующий репозиторий на github.

Задание по вариантам

№ варианта	Выборка	Размер исходного изображения	Оптимизатор
1	MNIST	28X28	SGD

Ход работы

Код программы:

```
import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
import matplotlib.pyplot as plt
import numpy as np

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Используемое устройство: {device}")

BATCH_SIZE = 64
EPOCHS = 10
LEARNING_RATE = 0.01
MOMENTUM = 0.9
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,))
])

train_dataset = datasets.MNIST(root="./data", train=True, download=True, transform=transform)
```

```
test_dataset = datasets.MNIST(root="./data", train=False, download=True, transform=transform)
```

```
train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True)
```

```
test_loader = DataLoader(test_dataset, batch_size=1000, shuffle=False)
```

```
class SimpleCNN(nn.Module):
```

```
    def __init__(self):
```

```
        super(SimpleCNN, self).__init__()
```

```
        self.conv1 = nn.Conv2d(in_channels=1, out_channels=16, kernel_size=3, padding=1)
```

```
        self.conv2 = nn.Conv2d(in_channels=16, out_channels=32, kernel_size=3, padding=1)
```

```
        self.pool = nn.MaxPool2d(kernel_size=2, stride=2)
```

```
        self.fc1 = nn.Linear(32 * 7 * 7, 128)
```

```
        self.fc2 = nn.Linear(128, 10)
```

```
        self.dropout = nn.Dropout(0.25)
```

```
    def forward(self, x):
```

```
        x = F.relu(self.conv1(x))
```

```
        x = self.pool(x)
```

```
        x = F.relu(self.conv2(x))
```

```
        x = self.pool(x)
```

```
        x = x.view(x.size(0), -1)
```

```
        x = F.relu(self.fc1(x))
```

```
        x = self.dropout(x)
```

```
        x = self.fc2(x)
```

```
        return x
```

```
model = SimpleCNN().to(device)
```

```
print(model)
```

```
criterion = nn.CrossEntropyLoss()
```

```
optimizer = optim.SGD(model.parameters(), lr=LEARNING_RATE, momentum=MOMENTUM)
```

```
train_losses = []
```

```
test_losses = []
```

```
test_accuracies = []
```

```
def train_epoch():
```

```
    model.train()
```

```
    running_loss = 0.0
```

```
    for images, labels in train_loader:
```

```
        images, labels = images.to(device), labels.to(device)
```

```
        optimizer.zero_grad()
```

```
        outputs = model(images)
```

```
        loss = criterion(outputs, labels)
```

```
        loss.backward()
```

```
        optimizer.step()
```

```

    running_loss += loss.item() * images.size(0)

epoch_loss = running_loss / len(train_loader.dataset)
return epoch_loss

def evaluate():
    model.eval()
    running_loss = 0.0
    correct = 0
    with torch.no_grad():
        for images, labels in test_loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            loss = criterion(outputs, labels)
            running_loss += loss.item() * images.size(0)

            preds = outputs.argmax(dim=1)
            correct += (preds == labels).sum().item()

    epoch_loss = running_loss / len(test_loader.dataset)
    accuracy = correct / len(test_loader.dataset)
    return epoch_loss, accuracy

print("Начинаем обучение...")
for epoch in range(1, EPOCHS + 1):
    train_loss = train_epoch()
    test_loss, test_acc = evaluate()

    train_losses.append(train_loss)
    test_losses.append(test_loss)
    test accuracies.append(test_acc)

    print(f"Эпоха {epoch}/{EPOCHS} | "
          f"Train loss: {train_loss:.4f} | "
          f"Test loss: {test_loss:.4f} | "
          f"Test accuracy: {test_acc*100:.2f}%")

plt.figure(figsize=(8, 5))
plt.plot(range(1, EPOCHS + 1), train_losses, label="Train loss")
plt.plot(range(1, EPOCHS + 1), test_losses, label="Test loss")
plt.xlabel("Эпоха")
plt.ylabel("Loss (CrossEntropy)")
plt.title("Изменение ошибки в процессе обучения (MNIST, SGD)")
plt.legend()
plt.grid(True)
plt.savefig("loss_curve.png")
plt.show()

print(f"\nИтоговая точность на тестовой выборке: {test accuracies[-1]*100:.2f}%")

```

```

def visualize_prediction(index=0):
    model.eval()
    image, true_label = test_dataset[index]
    input_tensor = image.unsqueeze(0).to(device)

    with torch.no_grad():
        output = model(input_tensor)
        probabilities = F.softmax(output, dim=1)
        predicted_label = probabilities.argmax(dim=1).item()
        confidence = probabilities.max().item()

    img_np = image.squeeze().numpy() * 0.3081 + 0.1307

    plt.figure(figsize=(4, 4))
    plt.imshow(img_np, cmap="gray")
    plt.title(f"Истинный класс: {true_label}\n"
             f"Предсказание: {predicted_label} (уверенность {confidence*100:.1f}%)")
    plt.axis("off")
    plt.savefig("prediction_example.png")
    plt.show()

random_index = np.random.randint(0, len(test_dataset))
visualize_prediction(random_index)

torch.save(model.state_dict(), "mnist_cnn_sgd.pth")
print("Модель сохранена в mnist_cnn_sgd.pth")

```

Результаты:

Используемое устройство: cpu

SimpleCNN(

(conv1): Conv2d(1, 16, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))

(conv2): Conv2d(16, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))

(pool): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)

(fc1): Linear(in_features=1568, out_features=128, bias=True)

(fc2): Linear(in_features=128, out_features=10, bias=True)

(dropout): Dropout(p=0.25, inplace=False)

)

Начинаем обучение...

Эпоха 1/10 | Train loss: 0.2658 | Test loss: 0.0638 | Test accuracy: 97.89%

Эпоха 2/10 | Train loss: 0.0688 | Test loss: 0.0406 | Test accuracy: 98.66%

Эпоха 3/10 | Train loss: 0.0488 | Test loss: 0.0318 | Test accuracy: 98.84%

Эпоха 4/10 | Train loss: 0.0392 | Test loss: 0.0311 | Test accuracy: 98.93%

Эпоха 5/10 | Train loss: 0.0331 | Test loss: 0.0305 | Test accuracy: 98.97%

Эпоха 6/10 | Train loss: 0.0296 | Test loss: 0.0272 | Test accuracy: 99.08%

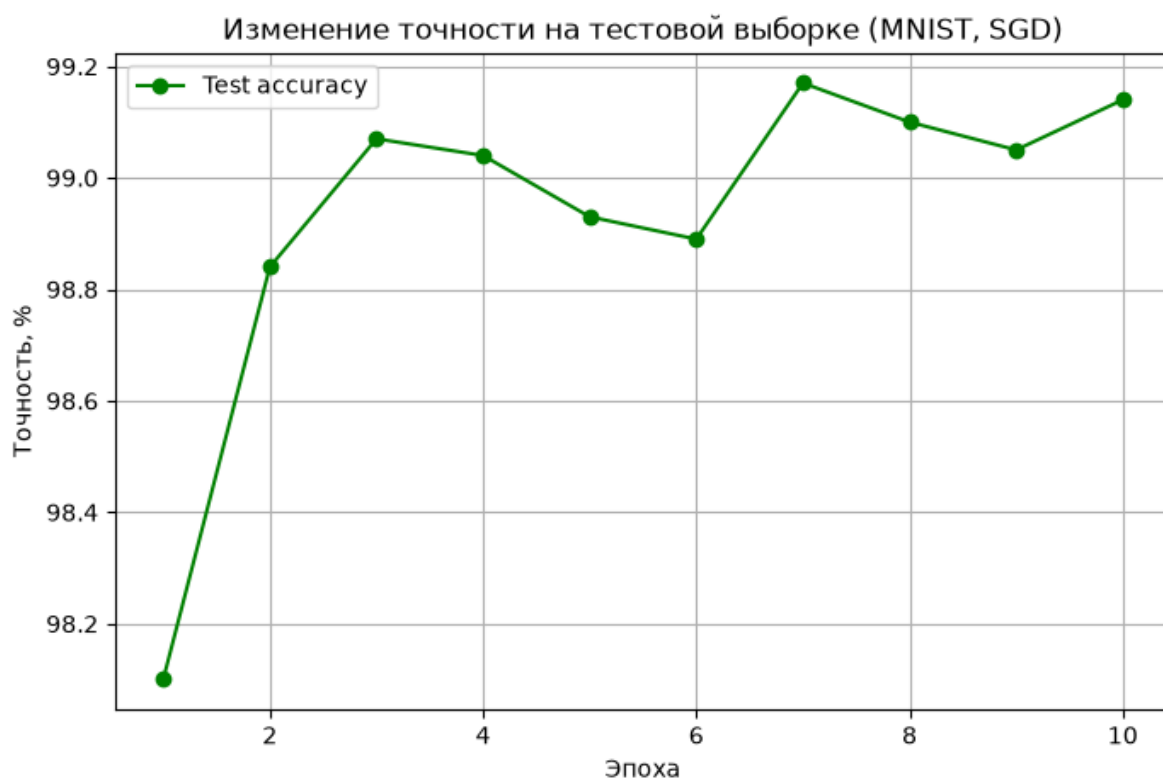
Эпоха 7/10 | Train loss: 0.0247 | Test loss: 0.0257 | Test accuracy: 99.15%

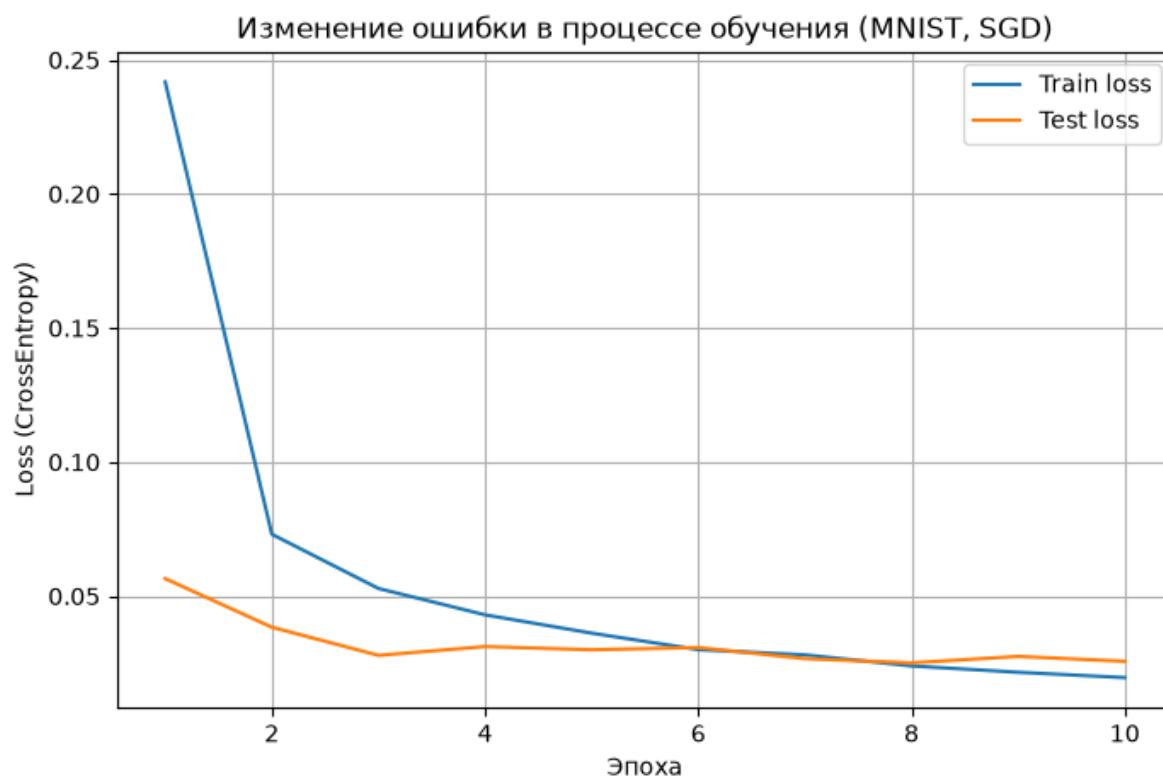
Эпоха 8/10 | Train loss: 0.0217 | Test loss: 0.0247 | Test accuracy: 99.21%

Эпоха 9/10 | Train loss: 0.0185 | Test loss: 0.0244 | Test accuracy: 99.20%

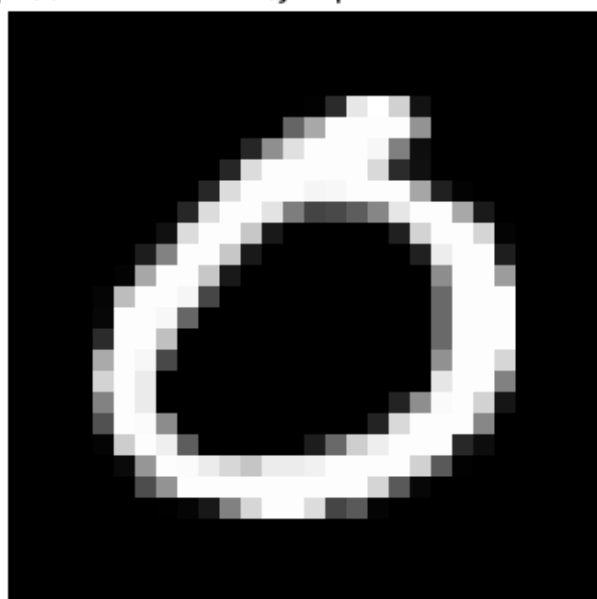
Эпоха 10/10 | Train loss: 0.0156 | Test loss: 0.0246 | Test accuracy: 99.20%

Итоговая точность на тестовой выборке: 99.20%





Истинный класс: 0
Предсказание: 0 (уверенность 100.0%)



Вывод: обучила простую CNN на MNIST с оптимизатором SGD и CrossEntropyLoss, итоговая точность на тестовой выборке составила 99.20%, что близко к современным state-of-the-art результатам для этой задачи. SGD сходилась медленно и стабильно, без резких скачков ошибки, а на визуализации сеть уверенно распознаёт цифры, ошибаясь в основном на похожих по написанию.